

Pizza Ordering Service

Dönem Projesi Final Sunumu

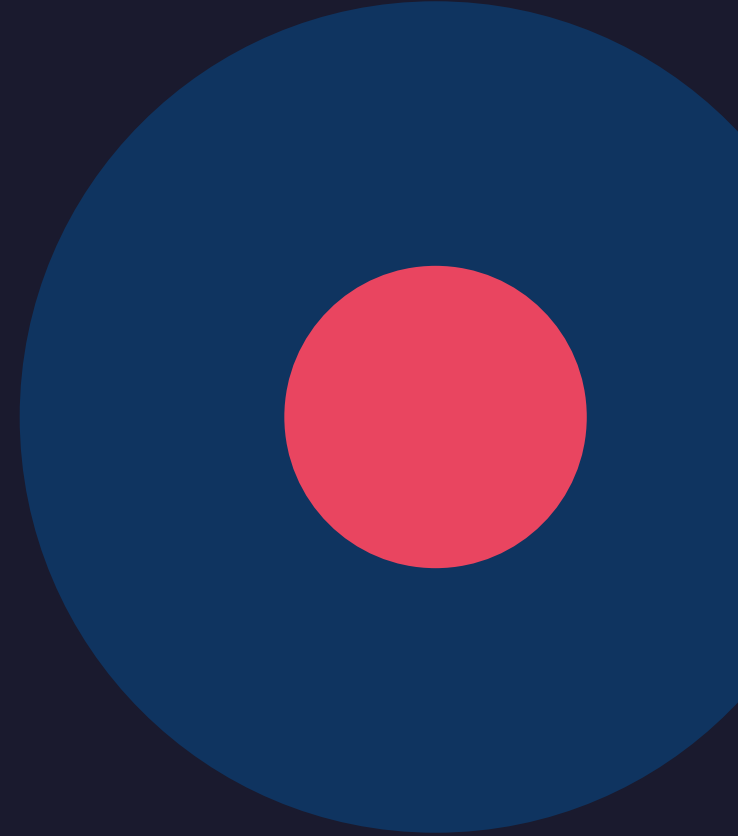
Bulut Mimarilerinde Test Mühendisliği | MTH2526-B25

Marmara Üniversitesi — Bilgisayar Mühendisliği

Yasir ARSLAN — 170423528

Büşra Ecem ÖZBEK — 170423966

Haziran 2026



Proje Hakkında

Ne yaptık? Neden pizza?

Amaç

- Karmaşık uygulama değil — endüstri standardında test ve dağıtım altyapısı kurmak.
- Domain: pizza siparişi — basit ama test edilebilir iş kuralları zengin.
- Bilinçli kapsam dışı: ödeme sistemi, kimlik doğrulama.

Teknoloji Yığını

Katman	Teknoloji
API	FastAPI + Python 3.12
Veritabanı	SQLAlchemy + PostgreSQL 16
AWS Emülasyon	LocalStack S3
Monitoring	Prometheus + Grafana
Tracing	OpenTelemetry + Jaeger
Konteyner	Docker Compose (7 servis)
Kubernetes	kind + KEDA + ArgoCD + Helm

Öne Çıkan İş Kuralları

- Durum makinesi: pending → preparing → ready → delivered
- Extra malzeme: her extra +\$2.50 (5 desteklenen)
- Kupon: PIZZA10 (%10), CHEESE20 (%20)
- Aynı siparişe 2. kupon uygulanamaz
- Durum atlama engellenir (pending→delivered = hata)

Mimari

Sistem bileşenleri ve aralarındaki ilişkiler



API Endpoint'leri

7 REST endpoint — iş kurallarıyla

Method	Endpoint	Açıklama	Önemli Kural
GET	/health	Sağlık kontrolü	CI smoke test + K8s probe
GET	/menu	Aktif menü listesi	is_available=True olanlar
POST	/orders	Sipariş oluştur	Fiyat hesabı + S3 arşivleme
GET	/orders	Tüm siparişler	Yeniden eskiye sıralı
GET	/orders/{id}	Sipariş detayı	Bulunamazsa 404
PATCH	/orders/{id}/status	Durum güncelle	pending→preparing→ready→delivered
POST	/orders/{id}/apply-coupon	Kupon uygula	İkinci kupon 400 hata

Fiyatlandırma Formülü

Toplam = (Birim Fiyat × Adet) + (2.50 × Extra Sayısı × Adet)

Örnek: *Margherita medium* (\$11.99) × 4 adet + *olive extra* × 4 = \$57.96

Test Stratejisi

18 test — %86 coverage — test piramidi

Unit Testler — 11 adet

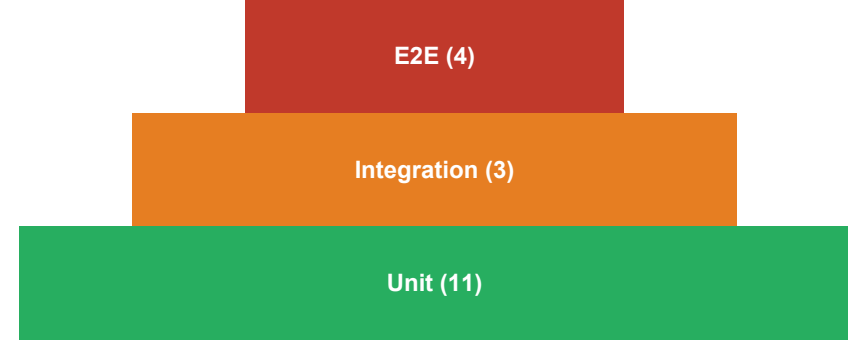
- SQLite in-memory — dış bağımlılık yok, 0.30 sn
- Geçersiz extra (pineapple) → 422
- Durum atlaması (pending→delivered) → 400
- Çift kupon → 400
- Sıralı durum geçişi doğrulaması
- S3 arşivleme tetiklenme kontrolü

Integration Testler — 3 adet

- Testcontainers: gerçek PostgreSQL 16 + LocalStack S3
- Sipariş DB'ye yazılıyor mu? Fiyat doğru mu?
- Kupon sonrası total_price güncellendi mi?

E2E Testler — 4 adet (Playwright Chromium)

- Gerçek tarayıcı — form doldur, sipariş oluştur
- Sipariş listele ve detay görüntüle
- Geçersiz extra → hata mesajı sayfada görünsün



Coverage: %86

(Şart: %70)

CI/CD Pipeline

GitHub Actions — 13 adım — her push'ta tetiklenir

Adım	İşlem	Araç
1–4	Checkout → Python 3.11 → Node.js → kubectl → kind cluster	GitHub Actions
5–6	Bağımlılıklar + Playwright Chromium kurulum	pip + playwright
7	Lint kontrolü	ruff check .
8	Testler + Coverage kontrolü	pytest --cov-fail-under=70
9–10	Docker image build + kind'a yükle	docker build + kind load
11–12	kubectl apply + rollout status bekleme (120s)	kubectl
13	Smoke test (curl /health) + Postman/Newman	curl + newman run

Bonus Katmanlar (+20 puan)

Bonus	Açıklama	Puan
Helm Chart	K8s kaynakları paketleni, values.yaml merkezi yönetim	+5
KEDA	Prometheus metriğine göre 1–5 replica otomatik ölçekleme	+5
ArgoCD	main branch merge → otomatik cluster deploy (GitOps)	+5
OpenTelemetry	FastAPI + SQL + AWS uçtan uca tracing — Jaeger'da görünür	+5

Monitoring ve Performans

Prometheus + Grafana + OpenTelemetry + Locust/k6

Grafana Dashboard — 3 Panel

Panel	Metrik	Ne Gösterir?
Request Throughput	rate(http_requests_total[1m])	Saniyedeki istek sayısı — trafik yoğunluğu
Error Rate	rate(http_requests_total{status=~'5..'}[1m])	HTTP 5xx oranı — sistem sağlığı
Latency P95	histogram_quantile(0.95, http_request_duration)	İsteklerin %95'inin yanıt süresi

Performans Testi

- k6 (perf/load-test.js) ve Locust (perf/locustfile.py) ile yük testi.
- Senaryo: /menu + POST /orders + GET /orders eş zamanlı yük.
- p95 latency ölçümü — sonuçlar perf/report.md'de.

OpenTelemetry Distributed Tracing

- FastAPI istekleri, SQLAlchemy sorguları ve S3 çağrıları otomatik trace edilir.
- Trace'ler Jaeger UI'da görüntülenebilir — localhost:16686.
- Bir isteğin API'den DB'ye, S3'e kadar tüm süreci tek ekranda takip edilir.

Sayılar ve Öğrendiklerimiz

Proje özeti

18

Test

%86

Coverage

7

Endpoint

7

Docker Servis

13

CI Adımı

+20

Bonus Puan

Öğrendiklerimiz

- Test katmanları: Unit hız sağlar, integration gerçekliği doğrular, E2E kullanıcı gözüyle bakar.

- Container teknolojileri: Docker Compose ile tekrar üretilebilir çok servisli ortam.

- Gözlemlenebilirlik: Prometheus + Grafana + OTEL üçlüsü birlikte nasıl çalışır.

- Docker kurulumunda WSL 2 gerekliliği — kurulum ek zaman aldı.

- Testcontainers Windows ortamında Docker Desktop yapılandırması gerektirdi.

- Kubernetes manifest bağımlılıkları başlangıçta karmaşık geldi.

Te ekk  rler

Sorularınız?